Analysis chinook database

Complex Joins and CTEs

```
# libraries
import sqlite3
import pandas as pd


# Connect to the SQLite database
conn = sqlite3.connect('/content/drive/MyDrive/sprints_tasks/Analysis  chinook database/da


conn
# test connection
print("connection successful")

 connection successful


# Get a cursor object cursor
= conn.cursor()

# Execute a query to get the table names
cursor.execute("SELECT name FROM sqlite_master WHERE type='table';")

# Fetch all the results
tables = cursor.fetchall()

# Print the table names for
table in tables:
   table[0]
   print(table[0])


  Album
      Artist
      Customer
      Employee
      Genre
      Invoice
      InvoiceLine MediaType
      Playlist
      PlaylistTrack Track


# Helper function to execute a query and return results as a DataFrame def
run_query(query):
      return pd.read_sql_query(query, conn)
```

Complex Joins and CTEs

```
# use INNER JOIN or LEFT JOIN to combine the tables
# use INNER JOIN as we only want customers who have invoices # create
a CTE to calculate the total spent per invoice
# aggregate the results to get the total spent per customer # order the
results and limit to the top 10
query1 = """
WITH InvoiceTotal AS (
    SELECT
        InvoiceId,
        SUM(UnitPrice * Quantity) AS TotalAmount FROM
    InvoiceLine
    GROUP BY InvoiceId
)
SELECT
    c.CustomerId,
    c.FirstName || ' ' || c.LastName AS CustomerName,
    SUM(it.TotalAmount) AS TotalSpent
FROM Customer c
INNER JOIN Invoice i ON c.CustomerId = i.CustomerId
INNER JOIN InvoiceTotal it ON i.InvoiceId = it.InvoiceId GROUP BY
c.CustomerId, CustomerName
ORDER BY TotalSpent DESC
LIMIT 10;
"""

result1 = run_query(query1) result1
```

	CustomerId	CustomerName	TotalSpent
0	6	Helena Holý	49.62
1	26	Richard Cunningham	47.62
2	57	Luis Rojas	46.62
3	45	Ladislav Kovács	45.62
4	46	Hugh O'Reilly	45.62
5	28	Julia Barnett	43.62
6	24	Frank Ralston	43.62
7	37	Fynn Zimmermann	43.62
8	7	Astrid Gruber	42.62
9	25	Victor Stevens	42.62

Window Functions for Ranking

```
# join InvoiceLine and Track tables
# calculate the total sales for each product # use
RANK() to rank products by total sales # identify the
top-selling products

query2 = """
WITH ProductSales AS (
      SELECT
            t.TrackId,
            t.Name AS ProductName,
            SUM(il.UnitPrice * il.Quantity) AS TotalSales FROM Track
      t
      INNER JOIN InvoiceLine il ON t.TrackId = il.TrackId GROUP
      BY t.TrackId, ProductName
)
SELECT
      TrackId,
      ProductName, TotalSales,
      RANK() OVER (ORDER BY TotalSales DESC) AS SalesRank FROM
ProductSales
ORDER BY SalesRank; """

result2 = run_query(query2)
print("Top 10 Selling Products:")
result2.head(10)
```

Top 10 Selling Products:

	TrackId	ProductName	TotalSales	SalesRank
0	2832	The Woman King	3.98	1
1	2850	The Fix	3.98	1
2	2868	Walkabout	3.98	1
3	3177	Hot Girl	3.98	1
4	3200	Gay Witch Hunt	3.98	1
5	3214	Phyllis's Wedding	3.98	1
6	3223	How to Stop an Exploding Man	3.98	1
7	3250	Pilot	3.98	1
8	2820	Occupation / Precipice	1.99	9
9	2821	Exodus, Pt. 1	1.99	9

```
# identify the CustomerId column in the Invoice table as a commonly queried column # create an
index on CustomerId
# write a query listing total sales for each customer # compare
query performance with and without the index
# optimized version of the query for better performance

# Create an index on the CustomerId column of the Invoice table
create_index_query = "CREATE INDEX IF NOT EXISTS idx_invoice_customerid ON Invoice(Custom
conn.execute(create_index_query)
conn.commit()
print("Index created successfully")
```

```
Index created successfully
```

```
# Query to list total sales for each customer (optimized using the index) optimized_query = """
SELECT
    c.CustomerId,
    c.FirstName || ' ' || c.LastName AS CustomerName,
    SUM(il.UnitPrice * il.Quantity) AS TotalSales
FROM Customer c
INNER JOIN Invoice i ON c.CustomerId = i.CustomerId
INNER JOIN InvoiceLine il ON i.InvoiceId = il.InvoiceId GROUP BY
c.CustomerId, CustomerName
ORDER BY TotalSales DESC; """

# Run and time the optimized query
print("\nRunning optimized query")
print("-"*60)
result3_optimized = run_query(optimized_query)
result3_optimized.head(10)
```

Running optimized query

	CustomerId	CustomerName	TotalSales
0	6	Helena Holý	49.62
1	26	Richard Cunningham	47.62
2	57	Luis Rojas	46.62
4	46	Hugh O'Reilly	45.62

```
# Remove index
remove_index = """
DROP INDEX idx_invoice_customerid; """
conn.execute(remove_index)
conn.commit()


 # Close the connection conn.close()
print("connection closed")
```